



COMPLETE EXECUTION BLUEPRINT

نظام تشخيص طبي بالذكاء الاصطناعي

دليل التنفيذ الكامل للمبتدئين — خطوة بخطوة

ملاحظة مهمة: هذا الدليل مكتوب خصيصًا لك كطالب مبتدئ يبيني المشروع ده لوحده لأغراض التعلم هتلاقي هنا كل حاجة محتاجها من أول ما تفهم الفكرة لحد ما تعمل عرض AMT. والحصول على شهادة من احترافي.



فهرس المحتويات

1. فهم المشروع بعمق
2. معمارية النظام الكاملة
3. خارطة الطريق والمراحل
4. خطة التعلم للمبتدئين
5. خطة التنفيذ الفردي
6. تصميم قاعدة البيانات
7. نظام Web Scraping
8. الكامل NLP Pipeline الـ
9. Machine Learning نماذج الـ
10. تدريب النماذج
11. Flask APIs تطوير الـ
12. نظام ترتيب الاحتمالات
13. تقييم النماذج
14. الاختبار وإصلاح الأخطاء

15. ال Deployment
 16. هيكل المجلدات الكامل
 17. GitHub Workflow
 18. MVP استراتيجية ال
 19. الجدول الزمني التفصيلي
 20. التحضير للعرض والشهادة
 21. الميزات المتقدمة
 22. قوائم التحقق النهائية
-
-

القسم الأول: فهم المشروع بعمق

ما هو المشروع ده بالضبط؟

تخيل إنك بتعمل دكتور ذكي على الإنترنت. المريض بيقولك "عندي صداع وحمى وكحة" وأنت بتقوله "على الأرجح عندك إنفلونزا (نسبة 75%) أو التهاب في الجهاز التنفسي (نسبة 60%) أو كوفيد (نسبة 40%) — وبتقوله كمان إيه اللي المفروض يعمل".

:المشروع بيعمل 3 حاجات أساسية

1. البريطاني NHS Inform يجمع بيانات طبية من موقع
↓
2. يتعلم منها ويبني نموذج ذكاء اصطناعي
↓
3. يستقبل أعراض ويتنبأ بالأمراض المحتملة

ليه المشروع ده مهم في الواقع؟

المشكلة الحقيقية: ملايين الناس كل يوم بتدور على جوجل لما تحس بأعراض — وجوجل مش بيديهم إجابات دقيقة. الناس بتقرأ مقالات عشوائية ومش بتعرف تفسرها.

بيقرأ الأعراض بشكل طبيعي زي ما بتتكلم، وببيديك احتمالات مرتبة من الأعلى للأدنى، **AI الحل اللي بتقدمه:** نظام مع تحذيرات لو في حاجة خطيرة محتاج تروح دكتور فيها فوراً.

كله بيشتغل إزاي؟ System ال

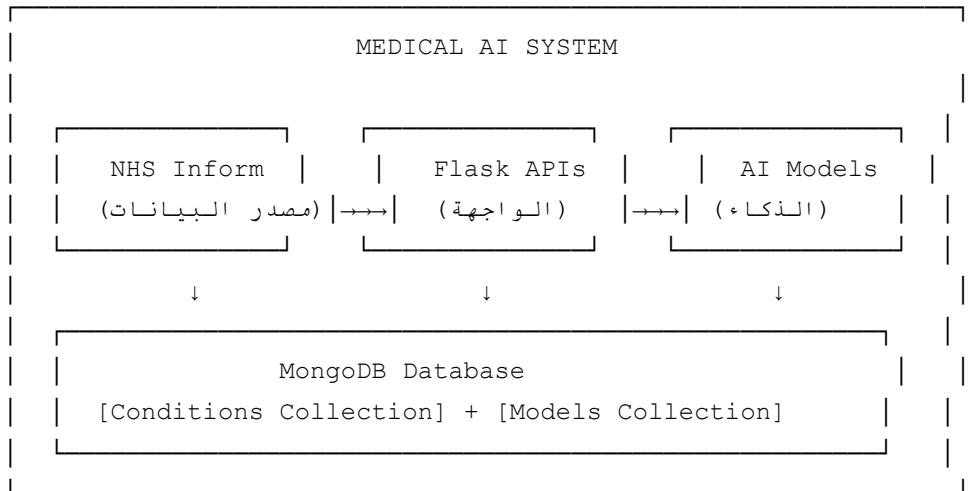
```
[المستخدم بيكتب أعراضه]
↓
[بتستقبل الطلب Flask API الـ]
↓
[بتنصف وتحلل الكلام NLP Pipeline الـ]
↓
[بيتنبأ بالأمراض AI Model الـ]
↓
[بيجيب التوصيات من قاعدة البيانات]
↓
[بيرجع للمستخدم قائمة مرتبة بالاحتمالات]
```

النهائي اللي هيطلع من المشروع Output ال

```
{
  "input_symptoms": "headache, fever, dry cough, fatigue",
  "patient_info": {"age": 25, "gender": "male"},
  "predictions": [
    {
      "condition": "Influenza",
      "probability": 0.78,
      "warnings": "Seek medical attention if fever exceeds 39°C",
      "recommendations": "Rest, drink fluids, paracetamol for fever"
    },
    {
      "condition": "COVID-19",
      "probability": 0.65,
      "warnings": "⚠️ Emergency: difficulty breathing → call 999",
      "recommendations": "Isolate, test immediately"
    },
    {
      "condition": "Common Cold",
      "probability": 0.42,
      "warnings": null,
      "recommendations": "Rest and stay hydrated"
    }
  ],
  "model_used": "BioBERT_v2",
  "confidence_score": 0.78
}
```

القسم الثاني: معمارية النظام الكاملة 🏗️

الصورة الكبيرة للنظام



Component: شرح كل

1. MongoDB — قاعدة البيانات

أو قاعدة بيانات عادية — يي حفظ Excel زي خزانة كبيرة بتحفظ فيها كل البيانات. مش زي MongoDB هو إيه؟ (Python ملفات زي القواميس في) JSON البيانات في شكل.

SQL؟ وملش MongoDB ليه

- البيانات الطبية مش لها شكل ثابت (الأعراض مختلفة لكل مرض)
- جديدة من غير ما تكسر حاجة fields سهل تضيف
- بيتعامل مع البيانات النصية الكبيرة كويس
- ML Models جّواه بيخليك تحفظ ملفات كبيرة زي ال GridFS ال

2. Flask APIs — الواجهة البرمجية

بياخده، Flask ال، (Request) زي ساعي بريد بين المستخدم والنظام. المستخدم بيبعت طلب Flask هو إيه؟ (Response) يعمل الشغل المطلوب، ويرجع رد.

رئيسية APIs عندنا 3

API 1: Data Handling API → NHS مسؤول عن جمع البيانات من
API 2: Preprocessing API → مسؤول عن تنظيف البيانات
API 3: Model API → مسؤول عن التدريب والتنبؤ

3. NLP Pipeline — معالجة اللغة

هو اللي بيخلي الكمبيوتر يفهم الكلام البشري (NLP (Natural Language Processing) هو إيه؟ الـ

```
"I have a bad headache and runny nose"
↓ Tokenization
["I", "have", "bad", "headache", "runny", "nose"]
↓ Cleaning (Remove stopwords)
["bad", "headache", "runny", "nose"]
↓ Lemmatization (كل كلمة لأصلها)
["bad", "headache", "runny", "nose"]
↓ Vectorization (تحويل لأرقام)
[0.2, 0.8, 0.1, 0.9, 0.0, 0.7, ...]
↓ AI Model
{"Influenza": 0.78, "Cold": 0.65}
```

4. Machine Learning Models — الذكاء الاصطناعي

هنبني 4 نماذج متدرجة في الصعوبة:

- **TF-IDF + Logistic Regression** — (ابدأ هنا)
- **RNN** — متوسط
- **LSTM** — متوسط-صعب
- **BERT/BioBERT** — الأصعب والأدق

5. Data Flow الكامل

```
1 NHS بيدخل على موقع Web Scraper : الخطوة
↓
الخطوة 2: يستخرج بيانات كل مرض (أعراض، أسباب، تحذيرات)
↓
MongoDB (Conditions Collection) الخطوة 3: يحفظها في
↓
Preprocessing API البيانات : الخطوة 4
↓
بتدرب النماذج على البيانات Model API : الخطوة 5
↓
MongoDB (GridFS) الخطوة 6: النماذج المدربة بتتحفظ في
```

↓
MongoDB الخطوة 7: لما مستخدم بيبيع أعراض، النموذج بيتحمّل من
↓
الخطوة 8: الأعراض بتتعالج وبيطلع التشخيص

القسم الثالث: خارطة الطريق والمراحل

المرحلة الأولى: التخطيط (الأسبوع الأول)

الهدف: تفهم المشروع كله قبل ما تكتب سطر كود واحد.

المهام:

- اقرأ هذا الدليل كله مرتين
- ارسم على ورقة كيف هيشغل النظام
- حدد المراحل اللي هتبدأ بيها
- خلي معاك ورقة بالمصطلحات وشرحها

المخرجات المتوقعة:

- خطة واضحة مكتوبة
- فهم كامل للمشروع

الوقت المقدر: 3-5 أيام

المرحلة الثانية: إعداد البيئة (الأسبوع الأول)

الهدف: تجهيز جهازك عشان يشتغل عليه.

المهام خطوة بخطوة:

```
# (لو مش موجود) Python الخطوة 1: تثبيت
# اختار 3.10 أو 3.11 - python.org من Download

# VS Code الخطوة 2: تثبيت
# code.visualstudio.com من Download
```

```
# الخطوة 3: إنشاء مجلد المشروع
mkdir medical-ai-project
cd medical-ai-project

# الخطوة 4: إنشاء virtual environment
python -m venv venv

# الخطوة 5: تفعيل الـ environment
# Windows:
venv\Scripts\activate
# Mac/Linux:
source venv/bin/activate

# الخطوة 6: تثبيت المكتبات الأساسية
pip install flask pymongo requests beautifulsoup4 scikit-learn pandas numpy

# الخطوة 7: تثبيت MongoDB
# Download من mongodb.com/try/download/community

# (واجهة رسومية) MongoDB Compass: تثبيت
# Download من mongodb.com/products/compass
```

المخرجات المتوقعة:

- جهاز جاهز للتطوير
- شغل محليًا MongoDB

الوقت المقدر: 1-2 يوم

(الأسبوع الثاني) Web Scraping: المرحلة الثالثة

NHS Inform. **الهدف:** جمع البيانات الطبية من

المهام:

- أساسيات BeautifulSoup تعلّم
- NHS Inform افهم هيكل موقع
- اكتب السكريبت
- اختبره على 5 أمراض الأول
- ثم شغله على الكل

الوقت المقدر: 5-7 أيام

المرحلة الرابعة: تصميم قاعدة البيانات (الأسبوع الثاني)

وتخزين البيانات MongoDB Collections الهدف: إنشاء

الوقت المقدر: 2-3 أيام

المرحلة الخامسة: تنظيف البيانات (الأسبوع الثالث)

الهدف: تنظيف وتجهيز البيانات للتدريب

الوقت المقدر: 5-7 أيام

(الأسبوع الرابع) TF-IDF المرحلة السادسة: النموذج الأساسي

يشتغل AI الهدف: بناء أول نموذج

الوقت المقدر: 5-7 أيام

(الأسبوع الخامس والسادس) Deep Learning المرحلة السابعة: نماذج

LSTM و RNN الهدف: بناء

الوقت المقدر: 10-14 يوم

(الأسبوع السابع والثامن) BERT و Transformers: المرحلة الثامنة

BioBERT أو Fine-tune BERT الهدف:

الوقت المقدر: 10-14 يوم

(الأسبوع التاسع) APIs المرحلة التاسعة: تطوير الـ

الكاملة Flask APIs الهدف: بناء الـ 3

الوقت المقدر: 7-10 أيام

المرحلة العاشرة: الاختبار والتقييم (الأسبوع العاشر)

الوقت المقدر: 5-7 أيام

(الأسبوع الحادي عشر) Deployment المرحلة الحادية عشرة: الـ

الوقت المقدر: 3-5 أيام

المرحلة الثانية عشرة: التوثيق والعرض (الأسبوع الثاني عشر)

الوقت المقدر: 5-7 أيام

القسم الرابع: خطة التعلم للمبتدئين 📚

ترتيب التعلم (مهم جدًا)

```
أساسيات Python : الأسبوع 1-2
↓
Git & GitHub : الأسبوع 3
↓
MongoDB : الأسبوع 4
↓
Flask : الأسبوع 5
↓
Web Scraping : الأسبوع 6
↓
NLP Basics : الأسبوع 7-8
↓
Machine Learning : الأسبوع 9-10
↓
Deep Learning : الأسبوع 11-12
↓
Transformers & BERT : الأسبوع 13-14
```

شرح كل موضوع بالتفصيل:

Python (الأساس)

Python. **ليه لازم:** كل المشروع مكتوب بـ

المواضيع الي تتعلمها:

```
# 1. المتغيرات والأنواع
name = "Ahmed"
age = 25
symptoms = ["headache", "fever", "cough"]

# 2. الدوال
def predict_disease(symptoms):
    # هنا نكتب كود التنبؤ
    return predictions

# 3. Classes (مهم جدًا للمشروع)
class MedicalModel:
    def __init__(self, model_name):
        self.name = model_name
        self.model = None

    def train(self, data):
        pass

    def predict(self, symptoms):
        pass

# 4. التعامل مع الملفات
import json
with open("data.json", "r") as f:
    data = json.load(f)

# 5. Error Handling
try:
    result = predict_disease(symptoms)
except Exception as e:
    print(f"Error: {e}")
```

الوقت: أسبوعين إذا بتذاكر يوميًا

أفضل مصادر مجانية:

- Python.org Tutorial

- CS50P (edX هارفارد — مجاني على)

Git & GitHub

ليه لازم: عشان تحفظ شغلك وترجع لأي نسخة قديمة لو حاجة اتكسرت

الأوامر الأساسية اللي هتحتاجها:

```
# جديد Repository إنشاء
git init

# إضافة الملفات
git add .

# حفظ التغييرات
git commit -m "Add web scraper"

# رفع على GitHub
git push origin main

# جديد Branch إنشاء (لـ features)
git checkout -b feature/nlp-pipeline

# الرجوع لـ main
git checkout main
```

الوقت: 3-5 أيام

MongoDB

ليه لازم: هو قاعدة بيانات المشروع

العمليات الأساسية:

```
from pymongo import MongoClient

# MongoDB الاتصال بـ
client = MongoClient("mongodb://localhost:27017/")
db = client["medical_ai_db"]

# Collections
conditions_col = db["conditions"]
models_col = db["models"]
```

```

# إضافة document
conditions_col.insert_one({
    "condition": "Influenza",
    "symptoms": ["fever", "cough", "fatigue", "headache"],
    "causes": ["Influenza virus type A or B"],
    "warnings": "Seek help if breathing is difficult",
    "recommendations": "Rest, fluids, paracetamol"
})

# البحث
result = conditions_col.find_one({"condition": "Influenza"})

# البحث بشرط
all_with_fever = conditions_col.find({"symptoms": {"$in": ["fever"]}})

# التحديث
conditions_col.update_one(
    {"condition": "Influenza"},
    {"$set": {"updated": True}}
)

```

الوقت: أسبوع

Flask

APIs. **ليه لازم:** عشان تبني الـ

مثال بسيط:

```

from flask import Flask, request, jsonify

app = Flask(__name__)

# Endpoint بسيط
@app.route("/predict", methods=["POST"])
def predict():
    # استقبال البيانات
    data = request.get_json()
    symptoms = data.get("symptoms", "")

    # هنا نكتب كود التنبؤ
    predictions = run_model(symptoms)

    # إرجاع النتيجة

```

```
return jsonify({
    "success": True,
    "predictions": predictions
})

if __name__ == "__main__":
    app.run(debug=True)
```

الوقت: أسبوع

NLP (معالجة اللغة الطبيعية)

المفاهيم الأساسية:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

# 1. Tokenization - تقطيع الجملة لكلمات
text = "I have severe headache and high fever"
tokens = word_tokenize(text)
# النتيجة: ['I', 'have', 'severe', 'headache', 'and', 'high', 'fever']

# 2. Stopword Removal - إزالة الكلمات غير المهمة
stop_words = set(stopwords.words('english'))
filtered = [w for w in tokens if w.lower() not in stop_words]
# النتيجة: ['severe', 'headache', 'high', 'fever']

# 3. Lemmatization - إرجاع الكلمة لأصلها
lemmatizer = WordNetLemmatizer()
lemmatized = [lemmatizer.lemmatize(w) for w in filtered]
# النتيجة: ['severe', 'headache', 'high', 'fever']

# 4. TF-IDF Vectorization - تحويل لأرقام
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus) # corpus = قائمة جمل
```

Machine Learning

المفاهيم الأساسية:

Training:

البيانات → النموذج يتعلم → نموذج مدرب

Testing:

بيانات جديدة → النموذج المدرب → تنبؤات

Evaluation:

metrics (accuracy, F1, etc.) → الإجابات الصحيحة vs التنبؤات

Deep Learning — RNN & LSTM

الفكرة:

- **RNN:** شبكة عصبية تتعامل مع التسلسلات (الجملة كلمة بكلمة)
- **LSTM:** بتتذكر المعلومات لفترة أطول RNN تطوير لـ

متى تستخدم:

- لما عندك نصوص طويلة
 - لما الترتيب مهم في الجملة
-

Transformers & BERT

الفكرة:

- **Transformer:** معمارية ثورية ظهرت 2017 — بتفهم الكلام بشكل أعمق بكثير
- **BERT:** على ملايين النصوص trained نموذج ضخمة
- **BioBERT:** على نصوص طبية — ده اللي هتستخدمه trained بس BERT نفس

مميز للمشروع ده BioBERT ليه:

- (ملايين الأبحاث الطبية) PubMed متدرب على
 - العادي BERT بيفهم المصطلحات الطبية أحسن من
 - دقته في تصنيف الأمراض أعلى
-
-

القسم الخامس: خطة التنفيذ الفردي

إيه اللي تعمله الأول؟

:الأولوية القصوى (لازم يشتغل)

- ✓ 1. Web Scraping من NHS
- ✓ 2. MongoDB حفظ البيانات في
- ✓ 3. NLP Pipeline
- ✓ 4. TF-IDF + Logistic Regression
- ✓ 5. Flask API أساسية

:الأولوية المتوسطة (مهم للشهادة)

- 6. LSTM Model
- 7. BERT/BioBERT
- 8. Probability Ranking
- 9. اختبار شامل

:اختياري (لو وقت)

- 10. Frontend
- 11. Deployment
- 12. Advanced Features

إيه الأجزاء الصعبة؟

الجزء	مستوى الصعوبة	الحل
BioBERT Fine-tuning	★★★★★	بسيط fine-tune واعمل pre-trained ابدأ بـ
Web Scraping بدون انقطاع	★★★	time.sleep() و error handling استخدم
GridFS (حفظ النماذج)	★★★	في كود جاهز هتلاقه في القسم 10
Multi-label Classification	★★★★	MultiLabelBinarizer استخدم من sklearn

إيه اللي ممكن تبسّطه؟

- لكل الموقع. ابدأ بـ 50-100 مرض scrape البيانات: مش لازم تعمل
- **Frontend:** demo بيكفي للـ Postman. مش لازم
- **Deployment:** للتسليم Render أو localhost. ابدأ بـ
- **Multiple Models:** بس (اثنين كافيين) BERT و TF-IDF ممكن تبدأ بـ

الأبسط MVP الـ

extras الإصدار اللي بيشتغل بدون أي MVP =

MVP: خطوات الـ

1. NHS لـ 50 مرض من scrape اعمل
2. نطّف البيانات
3. TF-IDF + Logistic Regression درّب
4. تاخذ أعراض وتدي تنبؤات Flask API اعمل
5. MongoDB احفظ النموذج في

يشتغل في أسبوعين MVP ده كافي كـ

القسم السادس: تصميم قاعدة البيانات

Collections في MongoDB

1. Conditions Collection

NHS. الغرض: تخزين كل المعلومات عن الأمراض المجموعة من

```
{
  "_id": ObjectId("..."),
  "condition": "Influenza",
  "url": "https://www.nhsinform.scot/illnesses-and-conditions/infections-and-pois",
  "symptoms": [
    "sudden fever",
    "aching body",
    "feeling exhausted",
    "dry cough",
    "sore throat",
    "headache",
    "difficulty sleeping"
  ],
  "causes": [
    "Influenza viruses type A and B",
    "Spreads through droplets when coughing or sneezing"
  ],
  "warnings": [
```



```
    "Seek urgent help if difficulty breathing",
    "Emergency if symptoms in child under 5 with high fever"
],
"recommendations": [
    "Rest at home",
    "Drink plenty of fluids",
    "Take paracetamol for fever and aches",
    "Do not give aspirin to children under 16"
],
"category": "Infections and poisoning",
"scraped_at": ISODate("2024-01-15T10:30:00Z"),
"source": "NHS Inform"
}
```

المقترحة Index:

```
# Index للبحث السريع على condition
conditions_col.create_index("condition", unique=True)

# Index للبحث في الأعراض symptoms على
conditions_col.create_index("symptoms")

# Text Index للنمى للبحث
conditions_col.create_index([("symptoms", "text"), ("causes", "text")])
```

2. Models Collection

الغرض: تخزين معلومات النماذج المدربة

```
{
  "_id": ObjectId("..."),
  "name": "bert_multilabel_v1",
  "type": "BioBERT",
  "version": "1.0",
  "gridfs_id": ObjectId("..."),
  "labels": [
    "Influenza",
    "COVID-19",
    "Common Cold",
    "Pneumonia",
    "Bronchitis"
  ],
  "metrics": {
    "accuracy": 0.87,
```

```

        "f1_score": 0.84,
        "top_3_accuracy": 0.93,
        "top_5_accuracy": 0.96,
        "recall": 0.82,
        "precision": 0.86
    },
    "training_config": {
        "epochs": 10,
        "batch_size": 16,
        "learning_rate": 2e-5,
        "optimizer": "AdamW",
        "train_size": 0.8,
        "test_size": 0.2
    },
    "training_data_size": 1250,
    "created_at": ISODate("2024-01-20T15:00:00Z"),
    "is_active": true
}

```

3. GridFS — تخزين ملفات النماذج

يمكن تكوين BERT النماذج الكبيرة في document لكل MB بتخزين البيانات في حدود 16 MongoDB: **الغرض** ويحفظها chunks بيقسم الملف لـ GridFS. أكبر من كده.

```

import gridfs
from pymongo import MongoClient
import pickle

# الاتصال
client = MongoClient("mongodb://localhost:27017/")
db = client["medical_ai_db"]
fs = gridfs.GridFS(db)

# حفظ نموذج في GridFS
def save_model_to_gridfs(model, model_name):
    # تحويل النموذج لـ bytes
    model_bytes = pickle.dumps(model)

    # حفظ في GridFS
    file_id = fs.put(model_bytes, filename=f"{model_name}.pkl")
    return file_id

# تحميل نموذج من GridFS
def load_model_from_gridfs(file_id):

```

```
model_bytes = fs.get(file_id).read()
model = pickle.loads(model_bytes)
return model
```

🕷 Web Scraping القسم السابع: نظام

NHS Inform فهم موقع

الموقع: <https://www.nhsinform.scot/illnesses-and-conditions/a-to-z/>

الموقع ده عنده

1. A-Z قائمة بكل الأمراض من **Index** صفحة الـ
2. **صفحة كل مرض**: فيها الأعراض والأسباب والتحذيرات والتوصيات

Web Scraper الكود الكامل للـ

```
# scraping/nhs_scraper.py

import requests
from bs4 import BeautifulSoup
import time
import logging
from pymongo import MongoClient
from datetime import datetime

# إعدادات logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('scraping.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)

class NHSScraper:

    def __init__(self, mongo_uri="mongodb://localhost:27017/"):

```

```

self.base_url = "https://www.nhsinform.scot"
self.az_url = f"{self.base_url}/illnesses-and-conditions/a-to-z/"
self.headers = {
    "User-Agent": "Mozilla/5.0 (Educational Project)"
}

# الاتصال بـ MongoDB
client = MongoClient(mongo_uri)
self.db = client["medical_ai_db"]
self.conditions_col = self.db["conditions"]

# لمنع التكرار Index
self.conditions_col.create_index("condition", unique=True)

def get_all_condition_links(self):
    """A-Z جلب كل روابط الأمراض من صفحة"""
    logger.info("Fetching A-Z index page...")

    try:
        response = requests.get(self.az_url, headers=self.headers)
        response.raise_for_status()
        soup = BeautifulSoup(response.text, "html.parser")

        links = []
        # البحث عن كل الروابط في قائمة الأمراض
        # (ده selector للموقع ونعدل الـ HTML هتحتاج تفحص الـ)
        for a_tag in soup.select("ul.az-page__list a"):
            href = a_tag.get("href")
            name = a_tag.text.strip()
            if href:
                full_url = self.base_url + href if href.startswith("/") else
                links.append({"name": name, "url": full_url})

        logger.info(f"Found {len(links)} conditions")
        return links

    except Exception as e:
        logger.error(f"Error fetching index: {e}")
        return []

def scrape_condition(self, url, condition_name):
    """استخراج بيانات مرض واحد"""
    try:
        response = requests.get(url, headers=self.headers)
        response.raise_for_status()
        soup = BeautifulSoup(response.text, "html.parser")

```

```

condition_data = {
    "condition": condition_name,
    "url": url,
    "symptoms": [],
    "causes": [],
    "warnings": [],
    "recommendations": [],
    "scraped_at": datetime.utcnow(),
    "source": "NHS Inform"
}

# استخراج المحتوى
# (دي هتتغير حسب هيكل الموقع الفعلي selectors الـ)

# Symptoms
symptoms_section = soup.find("h2", string=lambda t: t and "symptoms"
if symptoms_section:
    ul = symptoms_section.find_next("ul")
    if ul:
        condition_data["symptoms"] = [li.text.strip() for li in ul.fi

# Causes
causes_section = soup.find("h2", string=lambda t: t and "causes" in t
if causes_section:
    ul = causes_section.find_next("ul")
    if ul:
        condition_data["causes"] = [li.text.strip() for li in ul.find

# Warnings
warning_boxes = soup.find_all("div", class_=lambda c: c and "warning"
for box in warning_boxes:
    condition_data["warnings"].append(box.text.strip())

# Recommendations / Treatment
treatment_section = soup.find("h2", string=lambda t: t and
                                any(w in t.lower() for w in ["treatment",
if treatment_section:
    ul = treatment_section.find_next("ul")
    if ul:
        condition_data["recommendations"] = [li.text.strip() for li i

return condition_data

except Exception as e:
    logger.error(f"Error scraping {url}: {e}")
    return None

```

```

def save_to_mongodb(self, condition_data):
    """مع تجنب التكرار MongoDB حفظ في"""
    try:
        self.conditions_col.update_one(
            {"condition": condition_data["condition"]},
            {"$set": condition_data},
            upsert=True # لو مش موجود: أضفه، لو موجود: حدّثه
        )
        logger.info(f"✅ Saved: {condition_data['condition']}")
        return True
    except Exception as e:
        logger.error(f"❌ Error saving {condition_data['condition']}: {e}")
        return False

def run(self, limit=None, delay=2):
    """الكامل Scraper تشغيل"""
    logger.info("🚀 Starting NHS Scraper...")

    # جلب كل الروابط
    links = self.get_all_condition_links()

    if limit:
        links = links[:limit] # مرض N بس أول scrape للاختبار: اعمل

    success_count = 0
    fail_count = 0

    for i, link in enumerate(links, 1):
        logger.info(f"Processing {i}/{len(links)}: {link['name']}")

        # انتظر قبل كل طلب (لا تضغط على السيرفر)
        time.sleep(delay)

        # Scrape المرض
        data = self.scrape_condition(link["url"], link["name"])

        if data:
            saved = self.save_to_mongodb(data)
            if saved:
                success_count += 1
            else:
                fail_count += 1
        else:
            fail_count += 1

    logger.info(f"""
    ✅ Scraping Complete!

```

```

        Success: {success_count}
        Failed: {fail_count}
        Total: {len(links)}
    """

# تشغيل السكريبت
if __name__ == "__main__":
    scraper = NHSScraper()
    scraper.run(limit=50, delay=2) # ابدأ بـ 50 مرض للاختبار

```

Scraping: نصائح مهمة للـ

1. ما يحجبك NHS خلي في فترة انتظار بين كل طلب عشان موقع — `time.sleep(2)` استخدم
2. CSS selectors عشان تحدد الـ page source وشوف الـ NHS أول — افتح موقع HTML افحص الـ الصح
3. ابدأ بـ 5 أمراض — اختبر الكود على 5 أمراض الأول قبل ما تشغله على الكل
4. **Save Checkpoint** — لو الكود وقف scrape احفظ اسم آخر مرض عملت له

الكامل NLP Pipeline القسم الثامن: الـ

المراحل الكاملة من الأعراض للتشخيص

```

# preprocessing/nlp_pipeline.py

import re
import nltk
import spacy
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np

# تحميل الموارد
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

```

```

class MedicalNLPPipeline:

    def __init__(self):
        self.stop_words = set(stopwords.words('english'))
        self.lemmatizer = WordNetLemmatizer()

        # stopwords كلمات طبية مهمة مش هنحذفها حتى لو في قائمة الـ
        self.medical_important_words = {
            'no', 'not', 'without', 'severe', 'mild', 'moderate',
            'sudden', 'chronic', 'acute', 'high', 'low', 'very'
        }
        self.stop_words -= self.medical_important_words

    def clean_text(self, text):
        """تنظيف النص من الرموز والأحرف الغريبة"""
        if not text:
            return ""

        text = text.lower()
        text = re.sub(r'^a-z\s', ' ', text) # إزالة الأرقام والرموز
        text = re.sub(r'\s+', ' ', text) # إزالة المسافات الزائدة
        text = text.strip()

        return text

    def tokenize(self, text):
        """تقطيع النص لكلمات"""
        return word_tokenize(text)

    def remove_stopwords(self, tokens):
        """إزالة الكلمات غير المهمة"""
        return [t for t in tokens if t not in self.stop_words and len(t) > 1]

    def lemmatize(self, tokens):
        """إرجاع كل كلمة لأصلها"""
        return [self.lemmatizer.lemmatize(t) for t in tokens]

    def process(self, text):
        """كله Pipeline المعالجة الكاملة - الـ"""
        cleaned = self.clean_text(text)
        tokens = self.tokenize(cleaned)
        filtered = self.remove_stopwords(tokens)
        lemmatized = self.lemmatize(filtered)

        return " ".join(lemmatized)

    def process_batch(self, texts):

```



```

        """معالجة مجموعة نصوص دفعة واحدة"""
        return [self.process(t) for t in texts]

class SymptomPreprocessor:
    """معالجة أعراض المستخدم تحديدًا"""

    def __init__(self):
        self.pipeline = MedicalNLPPipeline()

        # قاموس المرادفات الطبية
        self.synonyms = {
            "headache": ["head pain", "head ache", "head hurts"],
            "fever": ["high temperature", "pyrexia", "febrile"],
            "cough": ["coughing", "hack"],
            "fatigue": ["tired", "exhausted", "weakness", "lethargy"],
            "nausea": ["feeling sick", "want to vomit"],
        }

    def expand_synonyms(self, text):
        """استبدال المرادفات بالكلمات الطبية الرسمية"""
        text_lower = text.lower()
        for standard_term, synonyms in self.synonyms.items():
            for syn in synonyms:
                if syn in text_lower:
                    text_lower = text_lower.replace(syn, standard_term)
        return text_lower

    def preprocess_symptoms(self, symptoms_text):
        """المعالجة الكاملة للأعراض"""
        # 1. استبدال المرادفات
        expanded = self.expand_synonyms(symptoms_text)

        # 2. تطبيق NLP Pipeline
        processed = self.pipeline.process(expanded)

        return processed

```

Machine Learning القسم التاسع: نماذج الـ

النموذج الأول: TF-IDF + Logistic Regression

تقيس عليه النماذج الثانية baseline **ليه تبدأ بيه؟** لأنه الأسهل، بيشتغل على أي جهاز، وهيديك

```
# models/tfidf_model.py

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, f1_score
import pickle
import numpy as np

class TfidfModel:

    def __init__(self):
        self.vectorizer = TfidfVectorizer(
            max_features=5000,      # أهم 5000 كلمة
            ngram_range=(1, 2),    # unigrams + bigrams
            min_df=2                # تجاهل الكلمات النادرة جدًا
        )
        self.classifier = OneVsRestClassifier(
            LogisticRegression(max_iter=1000, random_state=42)
        )
        self.mlb = MultiLabelBinarizer()
        self.is_trained = False

    def prepare_data(self, conditions_data):
        """تجهيز البيانات للتدريب"""
        X = [] # النصوص (الأعراض)
        y = [] # التصنيفات (أسماء الأمراض)

        for condition in conditions_data:
            symptoms_text = " ".join(condition.get("symptoms", []))
            if symptoms_text:
                X.append(symptoms_text)
                y.append([condition["condition"]]) # Multi-label format

        return X, y

    def train(self, X, y):
        """تدريب النموذج"""
        print("🇸🇦 Preparing labels...")
        y_encoded = self.mlb.fit_transform(y)

        print("✍️ Vectorizing text...")
```

```

X_vectorized = self.vectorizer.fit_transform(X)

print("🧠 Training model...")
self.classifier.fit(X_vectorized, y_encoded)

self.is_trained = True
print("✅ Training complete!")

def predict(self, symptoms_text, top_k=5):
    """التنبؤ بالأمراض"""
    if not self.is_trained:
        raise Exception("Model not trained yet!")

    # Vectorize
    X_vec = self.vectorizer.transform([symptoms_text])

    # الحصول على الاحتمالات
    probabilities = self.classifier.predict_proba(X_vec)[0]

    # ترتيب النتائج
    top_indices = np.argsort(probabilities)[::-1][:top_k]

    results = []
    for idx in top_indices:
        if probabilities[idx] > 0.05: # تجاهل الاحتمالات الصغيرة جدًا
            results.append({
                "condition": self.mlb.classes_[idx],
                "probability": round(float(probabilities[idx]), 4)
            })

    return results

def evaluate(self, X_test, y_test):
    """تقييم النموذج"""
    y_test_encoded = self.mlb.transform(y_test)
    X_test_vec = self.vectorizer.transform(X_test)

    y_pred = self.classifier.predict(X_test_vec)

    print(classification_report(
        y_test_encoded, y_pred,
        target_names=self.mlb.classes_
    ))

    f1 = f1_score(y_test_encoded, y_pred, average='weighted')
    print(f"\nWeighted F1 Score: {f1:.4f}")

```

```

        return f1

def save(self, filepath):
    """حفظ النموذج"""
    model_data = {
        "vectorizer": self.vectorizer,
        "classifier": self.classifier,
        "mlb": self.mlb
    }
    with open(filepath, 'wb') as f:
        pickle.dump(model_data, f)
    print(f"✅ Model saved to {filepath}")

def load(self, filepath):
    """تحميل النموذج"""
    with open(filepath, 'rb') as f:
        model_data = pickle.load(f)

    self.vectorizer = model_data["vectorizer"]
    self.classifier = model_data["classifier"]
    self.mlb = model_data["mlb"]
    self.is_trained = True
    print(f"✅ Model loaded from {filepath}")

```

النموذج الثاني: LSTM

```

# models/lstm_model.py

import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Embedding, LSTM, Dense, Dropout,
    Bidirectional, GlobalMaxPooling1D
)
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

class LSTMModel:

    def __init__(self, vocab_size=10000, max_len=200, embed_dim=128):
        self.vocab_size = vocab_size
        self.max_len = max_len
        self.embed_dim = embed_dim

```

```

self.tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
self.model = None
self.num_classes = None

def build_model(self, num_classes):
    """بناء معمارية الشبكة"""
    self.num_classes = num_classes

    self.model = Sequential([
        # Embedding layer: تحويل الكلمات لـ vectors
        Embedding(self.vocab_size, self.embed_dim, input_length=self.max_len)

        # Bidirectional LSTM: يقرأ الجملة من اليمين واليسار
        Bidirectional(LSTM(128, return_sequences=True)),

        # Dropout: لمنع الـ Overfitting
        Dropout(0.3),

        # LSTM ثاني
        Bidirectional(LSTM(64)),

        Dropout(0.3),

        # Dense layer
        Dense(128, activation='relu'),
        Dropout(0.2),

        # Output layer: sigmoid لأن ممكن أكثر من مرض في نفس الوقت
        Dense(num_classes, activation='sigmoid')
    ])

    self.model.compile(
        optimizer='adam',
        loss='binary_crossentropy', # Multi-label
        metrics=['accuracy']
    )

    print(self.model.summary())
    return self.model

def prepare_sequences(self, texts, labels=None, fit_tokenizer=False):
    """sequences تحويل النصوص لـ"""
    if fit_tokenizer:
        self.tokenizer.fit_on_texts(texts)

    sequences = self.tokenizer.texts_to_sequences(texts)
    padded = pad_sequences(sequences, maxlen=self.max_len, padding='post')

```

```

return padded

def train(self, X_train, y_train, X_val, y_val, epochs=20, batch_size=32):
    """تدريب النموذج"""

    callbacks = [
        # وقف التدريب لو النموذج مبقاش بيتحسن
        tf.keras.callbacks.EarlyStopping(
            patience=5, restore_best_weights=True
        ),
        # لو النموذج توقف عن التعلم learning rate خفض الـ
        tf.keras.callbacks.ReduceLROnPlateau(
            factor=0.5, patience=3
        )
    ]

    history = self.model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=epochs,
        batch_size=batch_size,
        callbacks=callbacks
    )

    return history

def predict(self, text, top_k=5):
    """التنبؤ"""
    sequence = self.prepare_sequences([text])
    probabilities = self.model.predict(sequence)[0]

    top_indices = np.argsort(probabilities)[::-1][:top_k]

    results = []
    for idx in top_indices:
        if probabilities[idx] > 0.1:
            results.append({
                "condition_index": int(idx),
                "probability": float(round(probabilities[idx], 4))
            })

    return results

def save(self, filepath):
    self.model.save(filepath)
    print(f"✅ LSTM model saved to {filepath}")

```

```
def load(self, filepath):
    self.model = tf.keras.models.load_model(filepath)
    print(f"✅ LSTM model loaded")
```

النموذج الثالث: BioBERT

```
# models/biobert_model.py

from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
import numpy as np

class BioBERTModel:

    def __init__(self, num_labels):
        self.num_labels = num_labels
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        print(f"Using device: {self.device}")

        # تحميل BioBERT pre-trained
        model_name = "dmis-lab/biobert-base-cased-v1.2"

        print("Loading BioBERT tokenizer...")
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)

        print("Loading BioBERT model...")
        self.model = AutoModelForSequenceClassification.from_pretrained(
            model_name,
            num_labels=num_labels,
            problem_type="multi_label_classification"
        )
        self.model.to(self.device)

    def tokenize(self, texts, max_length=256):
        """تحويل النصوص لـ tensors"""
        encodings = self.tokenizer(
            texts,
            truncation=True,
            padding=True,
            max_length=max_length,
            return_tensors="pt"
        )
        return encodings
```

```

def predict(self, symptoms_text, label_names, top_k=5):
    """التنبؤ بالأمراض"""
    self.model.eval()

    with torch.no_grad():
        inputs = self.tokenize([symptoms_text])
        inputs = {k: v.to(self.device) for k, v in inputs.items()}

        outputs = self.model(**inputs)
        logits = outputs.logits

        # احتمالات logits تحويل
        probabilities = torch.sigmoid(logits).cpu().numpy()[0]

    # ترتيب النتائج
    top_indices = np.argsort(probabilities)[::-1][:top_k]

    results = []
    for idx in top_indices:
        if probabilities[idx] > 0.1:
            results.append({
                "condition": label_names[idx],
                "probability": round(float(probabilities[idx]), 4)
            })

    return results

def save(self, save_path):
    self.model.save_pretrained(save_path)
    self.tokenizer.save_pretrained(save_path)
    print(f"✅ BioBERT saved to {save_path}")

```

مقارنة النماذج

النموذج	الدقة المتوقعة	الوقت للتدريب	RAM متطلبات	مناسب لـ
TF-IDF + LR	65-75%	دقائق	2GB	ابدأ هنا — MVP
LSTM	75-82%	ساعات	4GB	المرحلة الثانية
BioBERT	85-92%	ساعات-أيام	8GB+	المرحلة النهائية

الترتيب الأمثل للتنفيذ:

1. اعرفه شغال → TF-IDF ابدأ بـ
 2. حَسِّن الدقة → LSTM انتقل لـ
 3. أعلى دقة → BioBERT انهي بـ
-
-

Flask APIs القسم الحادي عشر: تطوير الـ

Data Handling API: الأولى API الـ

```
# api/data_handling_api.py

from flask import Flask, Blueprint, request, jsonify
from scraping.nhs_scraper import NHSScraper

data_bp = Blueprint('data', __name__)

@data_bp.route('/scrape', methods=['POST'])
def scrape_nhs():
    """
    Scraping بدء عملية الـ

    Request Body:
    {
        "limit": 50,          (اختياري - عدد الأمراض)
        "delay": 2            (اختياري - ثواني الانتظار بين الطلبات)
    }
    """
    try:
        data = request.get_json() or {}
        limit = data.get('limit', None)
        delay = data.get('delay', 2)

        scraper = NHSScraper()
        scraper.run(limit=limit, delay=delay)

        return jsonify({
            "success": True,
            "message": f"Scraping completed"
        }), 200
```

```

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

@data_bp.route('/conditions', methods=['GET'])
def get_conditions():
    """
    جلب كل الأمراض من MongoDB

    Query Params:
    - limit: (افتراضي 20) عدد الأمراض
    - skip: N تخطي أول
    - search: بحث بالاسم
    """
    try:
        from pymongo import MongoClient
        client = MongoClient("mongodb://localhost:27017/")
        db = client["medical_ai_db"]

        limit = int(request.args.get('limit', 20))
        skip = int(request.args.get('skip', 0))
        search = request.args.get('search', '')

        query = {}
        if search:
            query = {"condition": {"$regex": search, "$options": "i"}}

        conditions = list(
            db.conditions.find(query, {"_id": 0})
            .skip(skip)
            .limit(limit)
        )

        total = db.conditions.count_documents(query)

        return jsonify({
            "success": True,
            "total": total,
            "conditions": conditions
        }), 200

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

@data_bp.route('/conditions/<condition_name>', methods=['GET'])

```

```

def get_condition(condition_name):
    """جلب مرض واحد بالاسم"""
    try:
        from pymongo import MongoClient
        client = MongoClient("mongodb://localhost:27017/")
        db = client["medical_ai_db"]

        condition = db.conditions.find_one(
            {"condition": condition_name},
            {"_id": 0}
        )

        if not condition:
            return jsonify({"success": False, "error": "Condition not found"}), 4

        return jsonify({"success": True, "condition": condition}), 200

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

```

الثانية API ال: Preprocessing API

```

# api/preprocessing_api.py

from flask import Blueprint, request, jsonify
from preprocessing.nlp_pipeline import SymptomPreprocessor

preprocessing_bp = Blueprint('preprocessing', __name__)
preprocessor = SymptomPreprocessor()

@preprocessing_bp.route('/preprocess', methods=['POST'])
def preprocess_text():
    """
    تنظيف ومعالجة نص الأعراض

    Request Body:
    {
        "text": "I have severe headache and high fever"
    }

    Response:
    {
        "success": true,
        "original": "I have severe headache and high fever",

```

```

        "processed": "severe headache high fever"
    }
    """
    try:
        data = request.get_json()

        if not data or 'text' not in data:
            return jsonify({"success": False, "error": "Missing 'text' field"}),

        original_text = data['text']
        processed_text = preprocessor.preprocess_symptoms(original_text)

        return jsonify({
            "success": True,
            "original": original_text,
            "processed": processed_text,
            "word_count": len(processed_text.split())
        }), 200

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

@preprocessing_bp.route('/prepare-dataset', methods=['POST'])
def prepare_dataset():
    """
    للتدريب MongoDB الكامل من Dataset تجهيز الـ
    """
    try:
        from pymongo import MongoClient
        import json

        client = MongoClient("mongodb://localhost:27017/")
        db = client["medical_ai_db"]

        conditions = list(db.conditions.find({}, {"_id": 0}))

        dataset = []
        for condition in conditions:
            symptoms_text = " ".join(condition.get("symptoms", []))
            if symptoms_text:
                processed = preprocessor.preprocess_symptoms(symptoms_text)
                dataset.append({
                    "condition": condition["condition"],
                    "raw_symptoms": symptoms_text,
                    "processed_symptoms": processed,
                    "warnings": condition.get("warnings", []),

```

```

        "recommendations": condition.get("recommendations", [])
    })

    # حفظ الـ Dataset
    with open("data/processed_dataset.json", "w") as f:
        json.dump(dataset, f, indent=2)

    return jsonify({
        "success": True,
        "total_records": len(dataset),
        "message": "Dataset saved to data/processed_dataset.json"
    }), 200

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

```

الـ API الثالثة: Model API

```

# api/model_api.py

from flask import Blueprint, request, jsonify
import gridfs
from pymongo import MongoClient
import pickle
from datetime import datetime

model_bp = Blueprint('model', __name__)

client = MongoClient("mongodb://localhost:27017/")
db = client["medical_ai_db"]
fs = gridfs.GridFS(db)

# تخزين النموذج المحمل في الذاكرة
loaded_model = None
loaded_model_name = None
label_names = None

@model_bp.route('/train', methods=['POST'])
def train_model():
    """
    تدريب نموذج جديد

    Request Body:

```

```

{
    "model_type": "tfidf",    (tfidf, lstm, bert)
    "model_name": "tfidf_v1"
}
"""
try:
    data = request.get_json()
    model_type = data.get('model_type', 'tfidf')
    model_name = data.get('model_name', f'{model_type}_v1')

    # تحميل البيانات
    import json
    with open("data/processed_dataset.json") as f:
        dataset = json.load(f)

    X = [d['processed_symptoms'] for d in dataset]
    y = [[d['condition']] for d in dataset]
    labels = list(set([d['condition'] for d in dataset]))

    if model_type == 'tfidf':
        from models.tfidf_model import TFIDFModel
        from sklearn.model_selection import train_test_split

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42
        )

        model = TFIDFModel()
        model.train(X_train, y_train)
        f1 = model.evaluate(X_test, y_test)

        # حفظ في ملف مؤقت
        temp_path = f"/tmp/{model_name}.pkl"
        model.save(temp_path)

        # رفع على GridFS
        with open(temp_path, 'rb') as f_model:
            gridfs_id = fs.put(f_model.read(), filename=f"{model_name}.pkl")

    # Models Collection حفظ معلومات النموذج في
    db.models.insert_one({
        "name": model_name,
        "type": model_type,
        "gridfs_id": gridfs_id,
        "labels": labels,
        "metrics": {"f1_score": round(f1, 4)},
        "created_at": datetime.utcnow(),
    })

```

```

        "is_active": True
    })

    return jsonify({
        "success": True,
        "model_name": model_name,
        "f1_score": round(f1, 4),
        "message": "Model trained and saved to MongoDB"
    }), 200

else:
    return jsonify({"error": f"Model type '{model_type}' not implemented"})

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

@model_bp.route('/predict', methods=['POST'])
def predict():
    """
    التنبؤ بالأمراض من الأعراض

    Request Body:
    {
        "symptoms": "I have severe headache and high fever",
        "age": 25,
        "gender": "male",
        "model_name": "tfidf_v1",
        "top_k": 5
    }
    """
    global loaded_model, loaded_model_name, label_names

    try:
        data = request.get_json()

        if not data or 'symptoms' not in data:
            return jsonify({"success": False, "error": "Missing 'symptoms'"}), 400

        symptoms_text = data['symptoms']
        model_name = data.get('model_name', 'tfidf_v1')
        top_k = int(data.get('top_k', 5))
        age = data.get('age', None)
        gender = data.get('gender', None)

        # تحميل النموذج لو مش محمل
        if loaded_model_name != model_name:

```

```

print(f"Loading model: {model_name}")
model_info = db.models.find_one({"name": model_name})

if not model_info:
    return jsonify({"error": f"Model '{model_name}' not found"}), 404

model_bytes = fs.get(model_info['gridfs_id']).read()
loaded_model = pickle.loads(model_bytes)
loaded_model_name = model_name
label_names = model_info['labels']

# تنظيف الأعراض
from preprocessing.nlp_pipeline import SymptomPreprocessor
preprocessor = SymptomPreprocessor()
processed_symptoms = preprocessor.preprocess_symptoms(symptoms_text)

# التنبؤ
predictions = loaded_model.predict(processed_symptoms, top_k=top_k)

# إضافة التوصيات والتحذيرات من MongoDB
enriched_predictions = []
for pred in predictions:
    condition_data = db.conditions.find_one(
        {"condition": pred['condition']},
        {"_id": 0, "warnings": 1, "recommendations": 1}
    )

    enriched_predictions.append({
        **pred,
        "warnings": condition_data.get('warnings', []) if condition_data
        "recommendations": condition_data.get('recommendations', []) if c
    })

return jsonify({
    "success": True,
    "input": {
        "symptoms": symptoms_text,
        "processed": processed_symptoms,
        "age": age,
        "gender": gender
    },
    "predictions": enriched_predictions,
    "model_used": model_name,
    "top_k": top_k
}), 200

```

except Exception as e:


```

        return jsonify({"success": False, "error": str(e)}), 500

@model_bp.route('/models', methods=['GET'])
def list_models():
    """قائمة بكل النماذج المتاحة"""
    try:
        models = list(db.models.find(
            {},
            {"_id": 0, "gridfs_id": 0}
        ))

        # تحويل datetime لـ string
        for m in models:
            if 'created_at' in m:
                m['created_at'] = m['created_at'].isoformat()

        return jsonify({"success": True, "models": models}), 200

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

```

الرئيسية App ال

```

# app.py

from flask import Flask
from api.data_handling_api import data_bp
from api.preprocessing_api import preprocessing_bp
from api.model_api import model_bp

def create_app():
    app = Flask(__name__)

    # تسجيل Blueprints
    app.register_blueprint(data_bp, url_prefix='/api/data')
    app.register_blueprint(preprocessing_bp, url_prefix='/api/preprocessing')
    app.register_blueprint(model_bp, url_prefix='/api/model')

@app.route('/')
def home():
    return {
        "name": "Medical AI Diagnosis System",
        "version": "1.0",
    }

```

```

        "endpoints": {
            "data": "/api/data",
            "preprocessing": "/api/preprocessing",
            "model": "/api/model"
        }
    }

    return app

if __name__ == "__main__":
    app = create_app()
    app.run(debug=True, host="0.0.0.0", port=5000)

```

القسم الثاني عشر: نظام ترتيب الاحتمالات

```

# utils/probability_ranker.py

class ProbabilityRanker:

    EMERGENCY_KEYWORDS = [
        "difficulty breathing", "chest pain", "stroke",
        "heart attack", "unconscious", "severe bleeding",
        "anaphylaxis", "meningitis"
    ]

    def rank_predictions(self, predictions, patient_info=None):
        """
        ترتيب التنبؤات مع مراعاة عوامل الخطر
        """
        # تحقق من الطوارئ أولاً
        for pred in predictions:
            for warning in pred.get('warnings', []):
                if any(kw in warning.lower() for kw in self.EMERGENCY_KEYWORDS):
                    pred['is_emergency'] = True
                    pred['emergency_message'] = "⚠️ URGENT: Seek immediate medical attention"
                    break
            else:
                pred['is_emergency'] = False

        # ترتيب: الطوارئ أولاً، ثم حسب الاحتمال
        sorted_predictions = sorted(
            predictions,

```

```

        key=lambda x: (not x.get('is_emergency', False), -x['probability'])
    )

    # إضافة الترتيب
    for i, pred in enumerate(sorted_predictions, 1):
        pred['rank'] = i

    return sorted_predictions

def get_top_k(self, predictions, k=5):
    """تنبؤات K أعلى"""
    return predictions[:k]

```

القسم الثالث عشر: تقييم النماذج

المقاييس المهمة وشرحها ببساطة

1. Accuracy (الدقة الإجمالية)

عدد التنبؤات الصح / إجمالي التنبؤات = Accuracy

Accuracy = 85% → مثال: لو عملت 100 تنبؤ و صح منهم 85

وحدها مش كافية! لو 95% من الناس مش عندهم مرض نادر، نموذج بيقول "مفيش Accuracy: في الطب !
بدون ما يكون مفيد 95% accuracy مرض "دائمًا هيطلع

2. Recall (الحساسية) — الأهم في الطب!

التنبؤات الصح / (التنبؤات الصح + الحالات اللي فوّتها) = Recall

Recall = 80% → النموذج اكتشف 8 منهم X، مثال: 10 مريض بالمرض

مهم جدًا في الطب؟ لأن تفويت حالة مريضة أخطر بكثير من تشخيص خاطئ لحالة سليمة. أحسن Recall ليه
"تقول لشخص سليم "روح اتفحص" من إنك تقول لمريض "أنت كويس

3. F1-Score

$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$

ده المقياس الأفضل لأنه يوازن بين الاثنين

4. Top-K Accuracy

هل المرض الصح موجود في أول 3 تنبؤات؟ Top-3 Accuracy =

ده مهم لأن التشخيص الطبي مش دايماً يكون 100% واحد

كود التقييم الكامل

```
# evaluation/model_evaluator.py

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report
)
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

class ModelEvaluator:

    def __init__(self, model_name):
        self.model_name = model_name
        self.results = {}

    def evaluate_all_metrics(self, y_true, y_pred, y_pred_proba=None):
        """حساب كل المقاييس"""

        self.results = {
            "model_name": self.model_name,
            "accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred, average='weighted', zero_divis
            "recall": recall_score(y_true, y_pred, average='weighted', zero_divis
            "f1_score": f1_score(y_true, y_pred, average='weighted', zero_divisio
        }

        if y_pred_proba is not None:
            self.results["top_3_accuracy"] = self.top_k_accuracy(y_true, y_pred_p
            self.results["top_5_accuracy"] = self.top_k_accuracy(y_true, y_pred_p

        self.print_results()
        return self.results
```

```

def top_k_accuracy(self, y_true, y_pred_proba, k=3):
    """حساب Top-K Accuracy"""
    correct = 0
    for true_label, proba in zip(y_true, y_pred_proba):
        top_k_indices = np.argsort(proba)[::-1][:k]
        if true_label in top_k_indices:
            correct += 1
    return correct / len(y_true)

def print_results(self):
    """طباعة النتائج بشكل منظم"""
    print(f"\n{'='*50}")
    print(f"

```

القسم الرابع عشر: الاختبار وإصلاح الأخطاء

أنواع الاختبارات

1. Unit Tests

```
# tests/test_nlp_pipeline.py

import unittest
from preprocessing.nlp_pipeline import MedicalNLPPipeline

class TestNLPPipeline(unittest.TestCase):

    def setUp(self):
        self.pipeline = MedicalNLPPipeline()

    def test_clean_text(self):
        """اختبار تنظيف النص"""
        result = self.pipeline.clean_text("Hello! I have a fever... (38°C)")
        self.assertNotIn("!", result)
        self.assertNotIn("°", result)

    def test_tokenize(self):
        """اختبار التقطيع"""
        tokens = self.pipeline.tokenize("I have a headache")
        self.assertIn("headache", tokens)
        self.assertIsInstance(tokens, list)

    def test_full_pipeline(self):
        """اختبار Pipeline كامل"""
        result = self.pipeline.process("I have severe headache and high fever")
        self.assertIsInstance(result, str)
        self.assertIn("headache", result)
        self.assertIn("fever", result)
        self.assertNotIn("have", result) # تشالوت stopwords

    def test_empty_text(self):
        """اختبار النص الفاضي"""
        result = self.pipeline.process("")
        self.assertEqual(result, "")

if __name__ == '__main__':
    unittest.main()
```

2. API Testing بـ Postman

دي Requests بالـ Postman في Collection اعمل

1. POST <http://localhost:5000/api/data/scrape>
Body: {"limit": 10, "delay": 2}

2. GET `http://localhost:5000/api/data/conditions?limit=5`
3. POST `http://localhost:5000/api/preprocessing/preprocess`
Body: `{"text": "I have headache and fever"}`
4. POST `http://localhost:5000/api/model/train`
Body: `{"model_type": "tfidf", "model_name": "tfidf_v1"}`
5. POST `http://localhost:5000/api/model/predict`
Body: `{
 "symptoms": "I have headache, fever and cough",
 "age": 25,
 "gender": "male",
 "model_name": "tfidf_v1",
 "top_k": 5
}`

الأخطاء الشائعة وحلولها

الخطأ	السبب	الحل
<code>ConnectionRefusedError</code>	مش شغال MongoDB	ابدأ MongoDB service
<code>ModuleNotFoundError</code>	مكتبة مش مثبتة	<code>pip install [package]</code>
<code>404 Not Found</code>	URL غلطة في الـ	route افحص الـ
<code>Empty predictions</code>	نموذج مش متدرب	درب النموذج أول
<code>CUDA out of memory</code>	مش كفاية GPU RAM	<code>batch_size</code> قلّل الـ
<code>Scraping blocked</code>	IP حجب NHS	<code>time.sleep(5)</code> أضف

Deployment القسم الخامس عشر: الـ

Railway: أسهل طريقة للمبتدئين

Railway؟ له

- مجاني في البداية

- سهل جدًا
- Python و MongoDB يدعم

الخطوات:

```
# 1. إنشاء requirements.txt
pip freeze > requirements.txt

# 2. إنشاء Procfile
echo "web: python app.py" > Procfile

# 3. أول رفع GitHub على
git add .
git commit -m "Ready for deployment"
git push origin main

# 4. روح على railway.app
# جديد account اعمل
# New Project → Deploy from GitHub
# Repository اختار الـ
# تلقائيًا deploy هيعمل Railway
```

Docker (للمستوى المتقدم)

```
# Dockerfile

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]

# docker-compose.yml

version: '3.8'
services:
```



```
api:
  build: .
  ports:
    - "5000:5000"
  environment:
    - MONGO_URI=mongodb://mongo:27017/
  depends_on:
    - mongo

mongo:
  image: mongo:6
  ports:
    - "27017:27017"
  volumes:
    - mongo_data:/data/db

volumes:
  mongo_data:
```

القسم السادس عشر: هيكل المجلدات الكامل

```
medical-ai-project/
|
├──  api/                                # Flask APIs
|   ├── __init__.py
|   ├── data_handling_api.py          # جمع البيانات API
|   ├── preprocessing_api.py          # التنظيف API
|   └── model_api.py                  # النموذج والتنبؤ API
|
├──  scraping/                            # Web Scraping
|   ├── __init__.py
|   ├── nhs_scraper.py                # السكريبت الرئيسي
|   └── scraping.log                  # ملف السجلات
|
├──  preprocessing/                        # NLP
|   ├── __init__.py
|   └── nlp_pipeline.py               # الـ NLP Pipeline
|
├──  models/                                # نماذج الـ AI
|   ├── __init__.py
|   ├── tfidf_model.py                # TF-IDF + Logistic Regression
|   └── lstm_model.py                 # LSTM Model
```

```

├── biobert_model.py                # BioBERT Model
├──
├──  training/                        # Training Scripts
│   ├── train_tfidf.py
│   ├── train_lstm.py
│   └── train_biobert.py
├──
├──  evaluation/                    # التقييم
│   └── model_evaluator.py
├──
├──  utils/                        # أدوات مساعدة
│   ├── db_utils.py                # MongoDB helpers
│   ├── probability_ranker.py      # ترتيب الاحتمالات
│   └── gridfs_utils.py            # GridFS helpers
├──
├──  tests/                        # الاختبارات
│   ├── test_nlp_pipeline.py
│   ├── test_apis.py
│   └── test_models.py
├──
├──  data/                        # البيانات
│   ├── raw/                      # البيانات الخام
│   └── processed_dataset.json     # البيانات المعالجة
├──
├──  notebooks/                    # Jupyter Notebooks للتجارب
│   ├── 01_data_exploration.ipynb
│   ├── 02_nlp_experiments.ipynb
│   ├── 03_model_comparison.ipynb
│   └── 04_results_visualization.ipynb
├──
├──  docs/                        # التوثيق
│   ├── API_Documentation.md
│   ├── Architecture_Diagram.png
│   ├── Database_Schema.md
│   └── Evaluation_Report.md
├──
├──  deployment/                    # Deployment ملفات الـ
│   ├── Dockerfile
│   ├── docker-compose.yml
│   └── Procfile
├──
├── app.py                          # الرئيسية App الـ
├── requirements.txt                # المكتبات المطلوبة
├── config.py                       # الإعدادات
├── .gitignore
└── README.md

```

🐙 GitHub Workflow: القسم السابع عشر

Branches استراتيجية ال

```
main          ← الكود النهائي المستقر (لا تكتب فيه مباشرة)
↑
develop       ← الكود الجاري تطويره
↑
feature/scraping ← لما بتشتغل على الـ scraping
feature/nlp    ← لما بتشتغل على الـ NLP
feature/tfidf-model ← لما بتشتغل على النموذج
feature/lstm   ← LSTM
feature/bert   ← BERT
feature/apis   ← الـ APIs
```

الأوامر اليومية

```
# بداية يوم عمل جديد
git checkout develop
git pull origin develop

# الجديدة feature للـ branch إنشاء
git checkout -b feature/scraping

# بعد كل حاجة تشتغل
git add .
git commit -m "feat: add NHS scraper with error handling"

# رفع على GitHub
git push origin feature/scraping

# وشغالة تمام feature لو خلصت الـ
git checkout develop
git merge feature/scraping
```

احترافية Commit Messages كتابة

```
# الشكل العام :
```

[type]: [وصف قصير]

الأنواع:

feat: ميزة جديدة

fix: إصلاح bug

docs: تحديث التوثيق

refactor: تحسين الكود

test: إضافة اختبارات

أمثلة:

feat: add BioBERT model with multi-label classification

fix: resolve MongoDB connection timeout issue

docs: update API documentation with examples

test: add unit tests for NLP pipeline

MVP القسم الثامن عشر: استراتيجيات الـ

في 3 أسابيع MVP الـ

الأسبوع الأول:

(مرض 50) Web Scraping :يوم 1-2

MongoDB يوم 3-4: حفظ في

NLP Pipeline :يوم 5-7

الأسبوع الثاني:

تدريب + TF-IDF Model :يوم 8-11

الأساسية Flask APIs :يوم 12-14

الأسبوع الثالث:

يوم 15-17: ربط كل حاجة ببعض

يوم 18-19: اختبار

أساسية Documentation :يوم 20-21

!النتيجة: نظام شغال يتنبأ بالأمراض من الأعراض

القسم التاسع عشر: الجدول الزمني التفصيلي 17

خطة 3 أشهر لطالب مبتدئ

الشهر الأول: الأساسيات والبيانات

1 الأسبوع :

- Git & GitHub يوم 1-2: تعلم
- يوم 3-4: إعداد بيئة العمل
- أساسيات MongoDB يوم 5-7: تعلم

2 الأسبوع :

- BeautifulSoup يوم 1-3: تعلم
- NHS Scraper يوم 4-7: بناء

3 الأسبوع :

- على 50 مرض Scraper يوم 1-3: تشغيل
- أساسيات NLP يوم 4-7: تعلم

4 الأسبوع :

- NLP Pipeline يوم 1-4: بناء
- يوم 5-7: تنظيف البيانات

APIs الشهر الثاني: النماذج والـ

5-6 الأسبوع :

- TF-IDF Model بناء
- تدريبه وتقييمه
- أساسية Flask API

7-8 الأسبوع :

- Keras/TensorFlow تعلم
- LSTM Model بناء
- تدريبه

والإنهاء BERT: الشهر الثالث

9-10 الأسبوع :

- HuggingFace Transformers تعلم
- BioBERT Fine-tuning
- مقارنة النماذج

11 الأسبوع :

- الكاملة APIs ربط الـ
- Probability Ranking System
- اختبار شامل

12 الأسبوع :

- Deployment
- Documentation
- تحضير العرض

القسم العشرون: التحضير للعرض والشهادة

المفاهيم التي لازم تفهمها كويس

1. **Multi-label Classification:** الفرق بينه وبين Multi-class
2. **TF-IDF:** إزاي بيحسب أهمية الكلمات
3. **Attention Mechanism:** الفكرة الأساسية وراء BERT
4. **Overfitting vs Underfitting:** إزاي تتعرف عليه وتحله
5. **GridFS:** MongoDB ليه مش بتحفظ النموذج مباشرة في
6. **Top-K Accuracy:** العادية في الطب Accuracy ليه أهم من

أسئلة متوقعة وإجاباتها

العادي؟ BERT وملش BioBERT سؤال: ليه اخترت

وده بيخليه يفهم المصطلحات الطبية، PubMed متدرب على ملايين الأبحاث الطبية من BioBERT لأن "العادي متدرب على نصوص عامة مش متخصصة BERT. أحسن بكثير

وليه مهم؟ Top-K Accuracy سؤال: إيه هو

نتائج. في الطب ده مهم لأن الأعراض ممكن تكون K بيقيس هل المرض الصح موجود في أول Top-K "مشاركة بين أمراض كثير، والدكتور بيحتاج يشوف الاحتمالات الأعلى مش بس الأول

Imbalanced Dataset سؤال: إزاي تتعامل مع الـ

primary كـ Recall وكمان Logistic Regression في الـ class_weight parameter استخدمت "Accuracy بدل metric

SQL؟ وملش MongoDB سؤال: ليه

بيتعامل مع MongoDB. لأن البيانات الطبية غير متجانسة — كل مرض عنده أعراض مختلفة العدد والنوع "لحفظ ملفات النماذج الكبيرة GridFS البيانات الغير مرتبة أحسن، وعنده

خطة العرض (20 دقيقة)

دقائق: المقدمة 5

- ما هو المشروع؟
- ليه مهم؟
- كلمات بسيطة Architecture الـ

مباشر Demo :دقائق 5

- Postman من API call اعمل
- اعرض النتائج
- Probability Ranking اشرح الـ

دقائق: النتائج والمقارنة 5

- جدول مقارنة النماذج
- F1 Score رسوم بيانية للـ
- Insights أهم الـ

دقائق: التحديات والخلاصة 5

- أصعب حاجة واجهتها
- إيه اللي اتعلمته
- الخطوات الجاية

القسم الحادي والعشرون: الميزات المتقدمة ✨

الميزات الأساسية (لازم تكون موجودة)

- ✓ Web Scraping من NHS
- ✓ NLP Pipeline
- ✓ TF-IDF Model
- ✓ LSTM أو BERT
- ✓ Flask APIs الـ 3
- ✓ MongoDB + GridFS
- ✓ Probability Ranking
- ✓ Warnings & Recommendations

MVP بعد ال) الميزات الاختيارية المتقدمة

1. AI Chatbot
 - HuggingFace أو OpenAI API استخدم
 - المريض يتكلم مع بوت يسأله أعراضه
 2. Voice Input
 - speech_recognition library
 - المريض يقول أعراضه بدل ما يكتبها
 3. Explainable AI (XAI)
 - SHAP أو LIME
 - يشرح ليه النموذج قرر كده
 4. Arabic Language Support
 - AraBERT أو CAMEL Tools
 - دعم اللغة العربية
 5. Medical Dashboard
 - Streamlit أو React.js
 - واجهة مرئية جميلة
 6. Mobile App
 - React Native أو Flutter
 - تطبيق موبايل
-
-

القسم الثاني والعشرون: قوائم التحقق النهائية

قائمة التحقق من الكود

البيانات:

- ☐ Web Scraper شغال وبيجيب بيانات صحيحة
- ☐ MongoDB Collections متصمة صح
- ☐ Indexes مخطوطة
- ☐ scraper في ال Error handling
- ☐ تجنب التكرار في البيانات

الـ NLP:

- ☐ NLP Pipeline شغال
- ☐ Preprocessing API شغال

□ Synonym expansion شغل

النماذج:

- TF-IDF متدرب ومتقن
- LSTM (أو BioBERT) متدرب ومتقن
- GridFS النماذج محفوظة في
- مقارنة بين النماذج محتسبة

الـ APIs:

- Data Handling API شغلة
- Preprocessing API شغلة
- Model API شغلة
- Probability Ranking مطبق
- Warnings & Recommendations بترجع

الاختبار:

- Unit Tests للـ NLP
- API Testing في Postman
- Integration Test كامل

قائمة التحقق من التوثيق

- README.md واضح وكامل
- API Documentation (كل endpoint)
- Database Schema موثق
- Architecture Diagram مرسوم
- Evaluation Report مكتوب
- Code comments مكتوب فيه
- Requirements.txt محدث

قائمة التحقق من العرض

- Demo شغل على جهازك
 - Postman Collection جاهزة
 - النتائج مرئية (رسوم بيانية)
 - Slides (10-15 سلايد) جاهزة
 - أجبت على الأسئلة الشائعة
 - جرّبت العرض مرتين كاملتين
-
-

🏆 خلاصة ختامية

:المشروع ده هيعلمك

1. deployment كامل من جمع البيانات للـ AI كيفية بناء نظام
2. تطبيقي مش نظري بس NLP
3. (BERT/BioBERT) أحدث نماذج في معالجة اللغة
4. محترف Backend وكيفية بناء APIs
5. إدارة قواعد البيانات MongoDB
6. Git في الكود والتوثيق والـ Best Practices

:نصيحتين أخيرتين

.بيشتغل = مشروع ناجح. ثم طوّر خطوة بخطوة MVP النصيحة الأولى: "ابدأ صغير وكمل". الـ

Experimentation Report النصيحة الثانية: "السجل مهم". وثّق كل مشكلة وحلها — ده اللي بيميّز الـ الاحترافي.

🎓! بالتوفيق في مشروعاتك والحصول على الشهادة

Medical Diagnosis AI — AMT Certification تم إعداد هذا الدليل بناءً على وثيقة مشروع